Extended Lambda Calculus

A compiler into pure lambda calculus

2 + 3

```
(λm. λn. λf. λx. m f (n f x))
  (λf. λx. f (f x))
  (λf. λx. f (f (f x)))
```

Chasen Matsuoka
CS413 · Assignment 6

Project goal

SOURCE LANGUAGE

Numbers and booleans
Functions and recursion
Arithmetic and comparisons
Pairs and conditionals

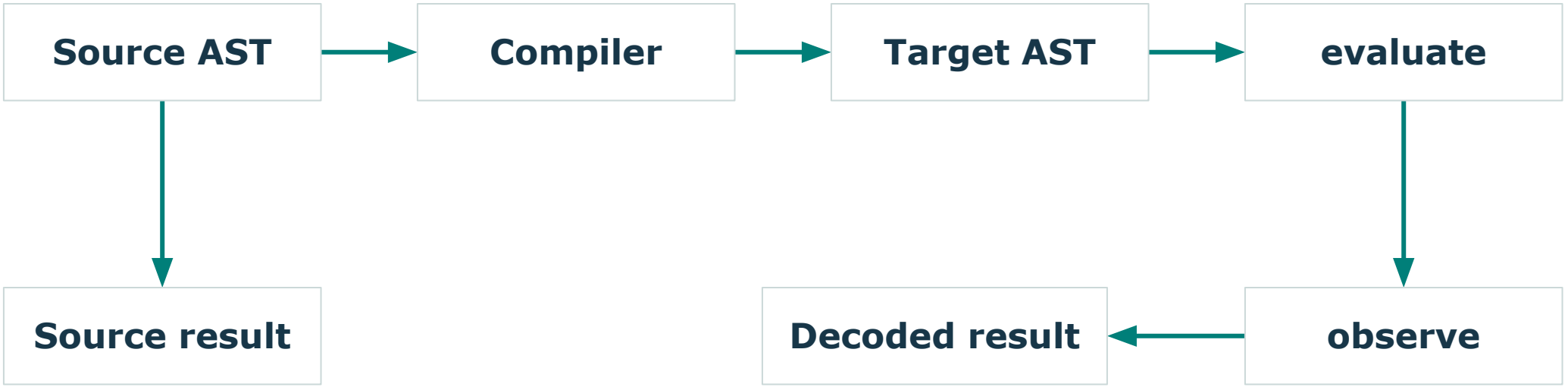
TARGET LANGUAGE

Var
Lam
App

2 + 3 compiles to ADD NUM(2) NUM(3)

Every abbreviation expands into pure lambda syntax.

System architecture



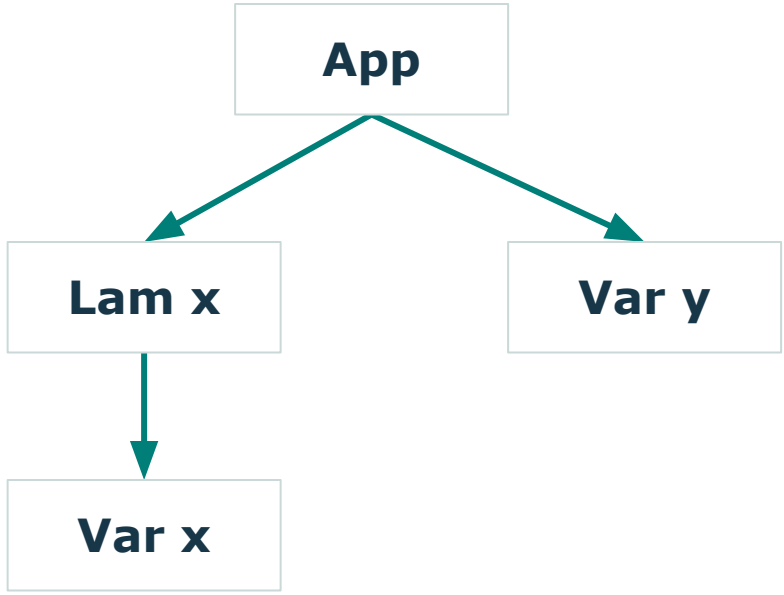
Shared evaluator, independent source semantics

Source result = decoded result = explicit expected answer

Target syntax and scope

Var(name)
Lam(name, body)
App(function, argument)

Immutable nodes
Validated syntax and identifiers
Open compilation, closed execution



((lam x. x) y)

Church numbers and booleans

Numbers represent repetition

$0 = \lambda f.\ \lambda x.\ x$
$3 = \lambda f.\ \lambda x.\ f\ (f\ (f\ x))$

Booleans represent selection

$\text{TRUE} = \lambda t.\ \lambda f.\ t$
$\text{FALSE} = \lambda t.\ \lambda f.\ f$

Arithmetic and predecessor

Pair transition

$$(a, b) \rightarrow (b, b + 1)$$

PRED selects the first component.
SUB repeats PRED and stops at zero.

Iterations	Pair state
0	(0, 0)
1	(0, 1)
2	(1, 2)
3	(2, 3)

LEQ m n = ISZERO (SUB m n)

Pairs and basic translation

PAIR = λa. λb. λs. s a b

Source	Target
λx. e	λx. C[e]
e1 e2	C[e1] C[e2]
let x = a in b	(λx. C[b]) C[a]
pair a b	PAIR C[a] C[b]

Both pair components evaluate before construction.

Evaluation strategy and budgets

Function first. Argument second. Then substitute.

$$(\lambda x.\ \text{body})\ \text{value} \ \rightarrow\ \text{body}[x := \text{value}]$$

Term	Budget	Outcome
I	0	Completed, 0 steps
I I	0	Exhausted, 0 steps
I I	1	Completed, 1 step

Only beta-value contractions count. Lambdas remain values.

Delayed conditionals and short-circuiting

(C[c] (λu. C[t]) (λv. C[f])) I

Wrap each branch in a lambda
Select one thunk, then force it with I

if true then 1 else LOOP → 1
false and LOOP → false
true or LOOP → true

Capture avoidance and fresh names

Substitute x := y into λy. x

Captured

λy. y

Capture avoided

λy_1. y

Rename conflicting binders and respect shadowing.
Reserve existing names before generating thunk parameters.

Recursion with Z

C[fix f(x). e] = Z (λf. λx. C[e])

Z = λf. (λx. f (λv. (x x) v))
(λx. f (λv. (x x) v))

The extra λv delays recursive self-application.

factorial 4 = 24

Fibonacci 5 = 5

Observing encoded results

Expected kind	Pure observation context
nat	Apply value to STEP and BASE
bool	Choose between two closed markers
pair	Project, then observe each component

STEP = λtail. λc. λz. c tail
BASE = λc. λz. z

Separate budget: completed, exhausted, or malformed

Testing and live demonstration

211

unit tests pass

14 / 14

required demonstrations pass

DEMO-08	Skip divergent branch	1
DEMO-10	Recursive factorial	24
DEMO-12	LOOP	Exhausted

```
python3 MySolution/demo.py --example DEMO-08 \
  --example DEMO-10 --example DEMO-12
```

Limitations and lessons

Limitations

Large expanded terms
Step budgets exclude traversal cost
Some Python recursion remains
No parser or source type checker

Lessons

Preserve variable scope
Delay unselected computation
Separate execution and observation

Tests support confidence. Exhaustion is a bounded outcome.